MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name: Adhiraj p

Roll No: 4

Class: CSA

Course: PCCST501 – Computer Networks

Module: 1

Website Chosen: Amazon (www.amazon.com)

Introduction: When a user types `www.amazon.com` into a browser's address bar and presses Enter, a beautifully coordinated sequence of events unfolds across several layers of the Internet's protocol stack. This process spans from resolving human-readable domain text to establishing bulletproof host connections and rendering graphic assets. This report presents a rigorous, structured analysis of each step involved in this transaction.

1. The TCP/IP Protocol Stack and the Role of Each Layer

Communication over the Internet is structured around the architectural framework of the TCP/IP model. This standard model compartmentalizes the vast complexities of network transport into four distinct operational layers. Each layer handles a singular set of tasks, processing and handing data down to the underlying medium when transmitting, or up to the application environment when receiving.

Application Layer (HTTP/HTTPS, DNS, FTP, SMTP)

Acts as the direct interface for user software. It structures raw web requests, processes system queries, and decodes content payload into renderable assets.

Transport Layer (TCP, UDP)

Manages endpoint-to-endpoint connectivity, slicing application data streams into segments, validating safe delivery, and overseeing error correction.

Internet Layer (IP, ICMP)

Defines structural packet routing across separate domains, stamping packets with standardized logical source and destination IP addresses.

Network Access Layer (Ethernet, Wi-Fi 802.11, ARP)

Handles physical infrastructure mapping, modulating binary packets into electrical, optical, or radio-frequency waveforms for local hardware transmission.

- **Application Layer:** The web browser initializes the request cycle here. It builds the explicit HTTP/HTTPS request and parses the returning HTML, CSS, and JavaScript. Domain name lookup via DNS is also executed within this layer.
- **Transport Layer:** Captures the application stream and divides it into sequentially numbered segments via TCP, ensuring complete, structured reconstruction at Amazon's servers.
- **Internet Layer:** Packages transport segments into network packets, calculating optimal delivery pathways across intermediate network routers using the target host's IP address.
- **Network Access Layer:** Maps logical IP packets into physical network frames, preparing data for structural transmission over physical local wires or wireless access point hardware.

2. The Role of DNS in Converting Domain Names to IP Addresses

Computers inherently route transactions using numeric network identifiers rather than textual domains. The Domain Name System (DNS) functions as a hierarchical distributed ledger to translate human-readable names like `www.amazon.com` into machine-routable IP strings:

1. **Cache Verification:** The browser queries its internal volatile memory and the local host operating system cache to see if the mapping for `www.amazon.com` is actively cached.
2. **ISP Resolver Query:** If absent, a recursive lookup query is sent to the local DNS Resolver (typically managed by the Internet Service Provider).
3. **Root Server Interrogation:** Lacking a cached record, the recursive resolver polls a global Root Nameserver (`.`), which parses the string and provides the address of the corresponding Top-Level Domain (TLD) Nameserver managing the `.com` registry.
4. **TLD Server Interrogation:** The TLD nameserver evaluates the address and forwards the resolver to Amazon's designated Authoritative Nameservers.
5. **Authoritative Resolution:** The Authoritative Nameserver issues the verified, active IP destination address (e.g., ***205.251.242.103***) back to the resolver, which stores it locally and serves it to the requesting browser environment.

3. How HTTP/HTTPS Requests and Receives Web Content

With an IP address established, the browser initiates application-level communication via the Hypertext Transfer Protocol (HTTP) or its cryptographically secure wrapper (HTTPS):

- **The Request:** The browser builds a structural `GET` command targeting Amazon's root directory. The structured text request provides critical server metrics:

```
GET / HTTP/1.1
Host: www.amazon.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Connection: keep-alive
```

- **The Response:** Amazon's infrastructure intercepts the request, processes the headers, and sends back an explicit server response message initiated by an HTTP status marker (e.g., `200 OK`). The body payload delivers the foundational index HTML code, prompting the browser to instantly parse it and make subsequent requests for stylesheets, media files, and dynamic scripts.

4. How TCP Provides Reliable Communication for HTTP

HTTP relies entirely on the underlying Transport Layer guarantees provided by the Transmission Control Protocol (TCP) to prevent corruption, loss, or out-of-order execution during transit:

- **The Three-Way Handshake:** To establish a reliable context before exchanging application data, a three-step greeting synchronizes state metrics:
  1. **SYN:** The client sends a synchronization packet containing an Initial Sequence Number (ISN).
  2. **SYN-ACK:** The server acknowledges the client's sequence marker and returns its own synchronization sequence flag.
  3. **ACK:** The client confirms the server's sequence flag, establishing a formal open channel.
- **Sequence Mapping and Reassembly:** TCP cuts large text packages into small chunks, appending an incremental index sequence number to each header block. The receiving device uses these numbers to sort segments properly, regardless of network arrival order.
- **Acknowledgements and Retransmissions:** Every arriving segment forces the destination host to return an explicit acknowledgement packet (ACK). If a chunk fails to arrive, the client stops acknowledging subsequent pieces, prompting the server to automatically retransmit the missing data block.

5. Application Architecture: Client-Server vs. Peer-to-Peer

The operational framework of Amazon's web platform adheres explicitly to a strict **Client-Server Architecture**.

Architectural Justification: The topology enforces an asymmetric operational balance. The **Client** (the end-user's web browser) acts strictly as a consumer, initiating connection requests only when prompted by a user action. The **Server** (Amazon's elastic AWS data center mesh) functions as a dedicated, always-on resource pool that listens for incoming connection requests and centrally serves secure payloads.

Unlike decentralized Peer-to-Peer (P2P) platforms where nodes share equal storage, upload, and structural responsibilities without a central registry, Amazon's central layout ensures total authority over customer profiles, financial transaction safety, tracking systems, and product database states.

6. Comparison Matrix: HTTP vs. FTP

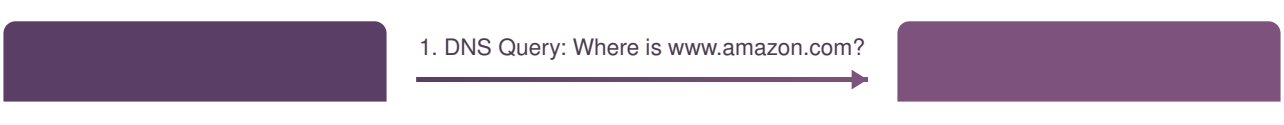
Parameter	HTTP (Hypertext Transfer Protocol)	FTP (File Transfer Protocol)
Primary Objective	Optimized for serving hyperlinked web assets and media structures (HTML, text, elements).	Designed specifically for bulk file movements between local hosts and storage nodes.
Connection Model	Uses a **single integrated connection** for passing both control parameters and data.	Employs **two parallel channels**: Port 21 for commands (Control) and Port 20 for file streams (Data).
Session Persistence	**Stateless;** each operational request is completely isolated from past exchanges.	**Stateful;** the host maintains an active session, tracking current working directories.
Transaction Profile	Engineered for high-frequency, short-lived transactional page resource queries.	Optimized for long-duration, high-volume upload and download processes.

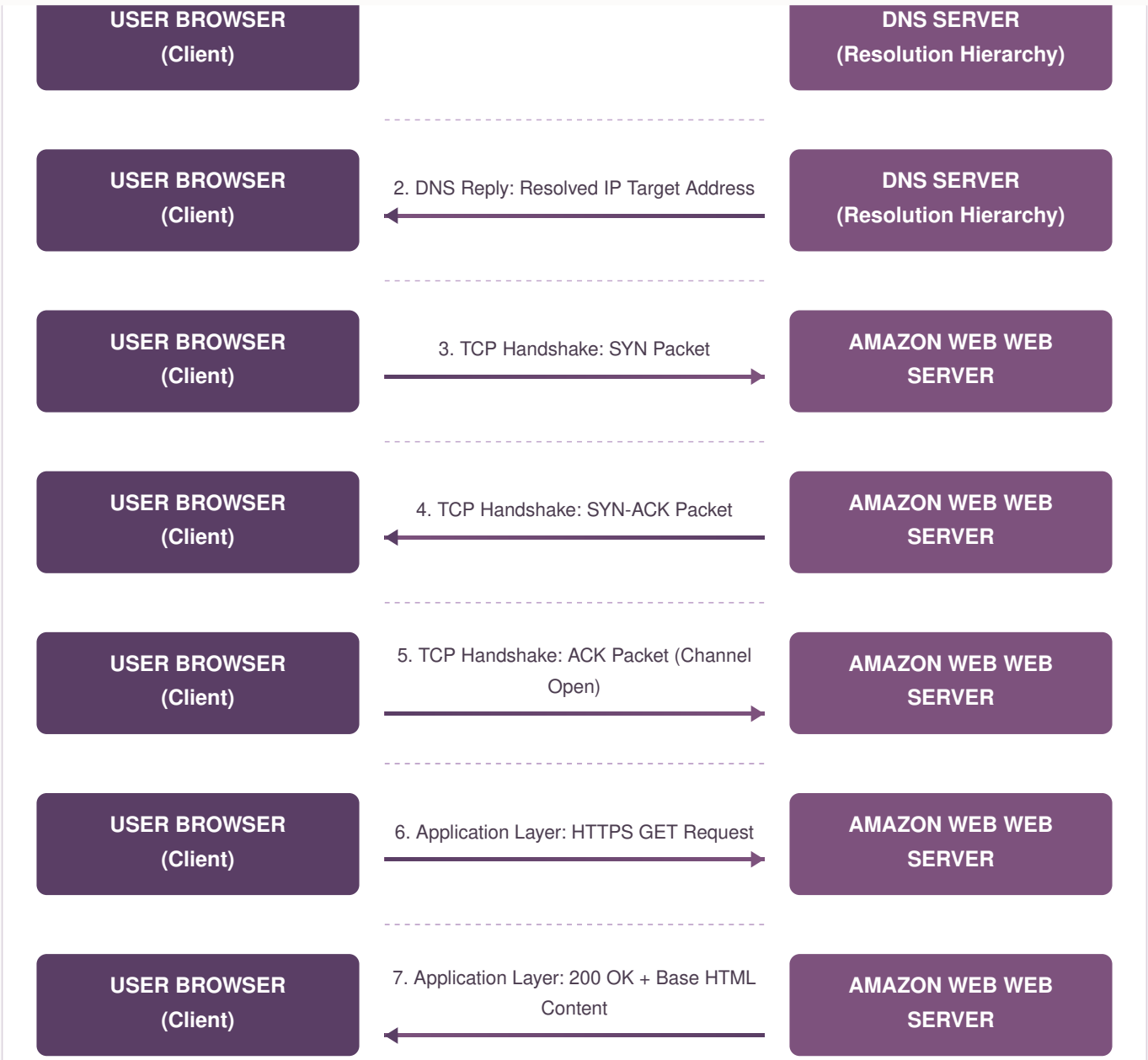
7. How Cookies Improve User Experience

A cookie is a lightweight alphanumeric data string sent by a web server and stored locally within a user's browser sandbox environment. Because the underlying HTTP standard is entirely stateless, servers cannot naturally remember identities across page clicks. Cookies provide a persistent state mechanism to bridge this gap.

Real-World Application (Amazon Shopping Cart): When a user shops on Amazon without authenticating an account, the server generates a unique tracking session ID string inside a local browser cookie. If the user adds a product to the cart and navigates to another product or closes the browser window completely, the browser attaches that cookie token to every subsequent request header. Amazon's backend identifies the token, links it to its active cart database, and preserves the user's shopping state. Without cookies, the cart would clear completely with every mouse click.

8. Complete Communication End-to-End Flow Diagram





9. Conclusion

Application layer protocols serve as the fundamental bridge linking human digital requests to deep underlying network routing architectures. Protocols such as HTTP/HTTPS and DNS isolate end users from the technical complexities of bit routing, logical network addressing, and physical wave propagation by presenting clear, uniform, and reliable programmatic interfaces. In daily operations, these protocols work quietly in the background to ensure that actions like loading a global marketplace, streaming video assets, or exchanging sensitive text happen securely and instantaneously. Without these standardized application layers, the modern Internet would lack universal compatibility, fragmenting web access and diminishing structural efficiency for users worldwide.